Task 1 Byte Pair Encoding algorithm

Initialize corpus

First, define the words and their frequencies. Each word is treated as a separate sequence. Word frequencies = weights:

low: 5

lower: 2

lowest: 3

newer: 3

wider: 2

widest: 3

Turn 0 – Initial tokenization

Split every word into individual characters.

Vocabulary: [d, e, i, l, n, o, r, s, t, w]

Tokenized corpus: [5(l o w), 2(l o w e r), 3(l o w e s t), 3(n e w e r), 2(w i d e r), 3(w i d e s t)]

Now count all adjacent character pairs. Each pair is weighted by the frequency of ist word.

Turn1 – 1st merge

Pair

low: 5(lo) 5(ow)

lower: 2(lo) 2(ow) 2(we) 2(er)

lowest: 3(lo) 3(ow) 3(we) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(we) 3(er)

wider: 2(wi) 2(id) 2(de) 2(er)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

Add the frequencies of identical pairs:

most frequent pairs: 10(lo) 10(ow) 8(we) 7(er) 6(es) 6(st) 5(de) 5(id) 5(wi) 3(ew) 3(ne)

The highest Frequency ist 10. There is a tie between (lo) and (ow).

According to the alphanetical rule (lo) is selected.

Merge 1 : (**l o) -> lo**

Add token to vocabulary and merge the selected pair in every word.

Turn 2 – 2nd merge

Vocabulary: [d, e, i, l, lo , n, o, r, s, t, w] *# vocabulary grows with every new token*

Updated tokenized corpus:

[5(lo w), 2(lo w e r), 3(lo w e s t), 3(n e w e r), 2(w i d e r), 3(w i d e s t)]

Reinitialize the pair frequency dictionary and count all adjacent pairs again.

Pair frequency calculation:

low: 5(low)

lower: 2(low) 2(we) 2(er)

lowest: 3(low) 3(we) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(we) 3(er)

wider: 2(wi) 2(id) 2(de) 2(er)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

most frequent pairs: 10(low) 8(we) 7(er) 6(es) 6(st) 5(de) 5(id) 5(wi) 3(ew) 3(ne)

Highest frequency is 10.

Merge 2: (lo w) -> low

Update the vocabulary and the tokenized corpus (merge).

Turn 3 – 3rd merge

Vocabulary: [d, e, i, l, lo, low, n, o, r, s, t, w]

Updated tokenized corpus:

[5(low), 2(low e r), 3(low e s t), 3(n e w e r), 2(w i d e r), 3(w i d e s t)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again using the updated sequences. (# low is finished)

Pair frequency calculation:

lower: 2(lowe) 2(er)

lowest: 3(lowe) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(we) 3(er)

wider: 2(wi) 2(id) 2(de) 2(er)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

most frequent pairs: 7(er) 6(es) 6(st) 5(de) 5(id) 5(low) 5(lowe) 5(wi) 3(ew) 3(ne)

Highest frequency is 7.

Merge 3: (e r) -> er

Update the vocabulary and the tokenized corpus.

Turn 4 - 4th merge

Vocabulary: [d, e, er, i, l, lo, low, n, o, r, s, t, w]

updated tokenized corpus:

[5(low), 2(low er), 3(low e s t), 3(n e w er), 2(w i d er), 3(w i d e s t)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again

Pair frequency calculation:

lower: 2(lower)

lowest: 3(lowe) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(wer)

wider: 2(wi) 2(id) 2(der)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

most frequent pairs: 6(es) 6(st) 5(id) 5(wi) 3(de) 3(ew) 3(lowe) 3(ne) 3(wer) 2(der) 2(lower)

Highest frequency is 6. Tie betwenn (es) and (st). According to the alphabetical rule (es) is selected.

Merge 4: (e s) -> es

Update the vocabulary and the tokenized corpus.

Turn 5 – 5th Merge

Vocabulary: [d, e, er, es, i, l, lo, low, n, o, r, s, t, w]

updated tokenized corpus:

[5(low), 2(low er), 3(low es t), 3(n e w er), 2(w i d er), 3(w i d es t)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again

Pair frequency calculation:

lower: 2(lower)

lowest: 3(lowes) 3(est)

newer: 3(ne) 3(ew) 3(wer)

wider: 2(wi) 2(id) 2(der)

widest: 3(wi) 3(id) 3(des) 3(est)

most frequent pairs: 6(est) 5(id) 5(wi) 3(des) 3(ew) 3(lowes) 3(ne) 3(wer) 2(der) 2(lower)

highest frequency is 6.

Merge 5: (es t) -> est

Update the vocabulary and the tokenized corpus.

Turn 6 – 6th Merge

Vocabulary: [d, e, er, es, est, i, l, lo, low, n, o, r, s, t, w]

updated tokenized corpus:

tokenized [5(low), 2(low er), 3(low est), 3(n e w er), 2(w i d er), 3(w i d est)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again

Pair frequency calculation:

lower: 2(lower)

lowest: 3(lowest)

newer: 3(ne) 3(ew) 3(wer)

wider: 2(wi) 2(id) 2(der)

widest: 3(wi) 3(id) 3(dest)

most frequent pairs: 5(id) 5(wi) 3(dest) 3(ew) 3(lowest) 3(ne) 3(wer) 2(der) 2(lower)

Higehest frequency is 5. Tie betwenn (id) and (wi). According to the alphabetical rule (id) is selected.

Merge 6: (i d) -> id

Update the vocabulary and the tokenized corpus.

Final Result

Final Vocabulary: [d, e, er, es, est, i, id, l, lo, low, n, o, r, s, t, w]

finalized tokenized corpus:

[5(low), 2(low er), 3(low est), 3(n e w er), 2(w id er), 3(w id est)]

Six merge operations:

1. l o -> lo
2. lo w -> low
3. e r -> er
4. e s -> es
5. es t -> est
6. i d -> id

Tokenize unseen word *lowers*

Initialize on character level

[l o w e r s]

Apply the learned BPE Merges in order

1. l o -> lo
2. lo w -> low
3. e r -> er
4. ! e s -> es not possible because *e* is no longer a seperate token. It already belongs to *er*.
5. ! e s t -> est not possible
6. ! i d -> id not possible

Final tokenization: [low er s]

<u>Reflection:</u>

(explanation: Downstream NLP means here applying learned tokenizer of an unseen word (lowers) after the upstream training phase with the corpus)

- The learned tokenization partly reflects the morphologically structure of the words.
- It works for [low], [low er], [low est] and [low er s] where *low* represents the word stem (lexical/semantic meaning) and *er, est, s* are meaningfull suffixes (grammatical meaning) – they are all "real morphemes" althoug they where only merged by counting frequencies.
- For [n e w er], [w id er] and [w id est] the tokenization only partly refelcts meaningful morpheme structures. Here, only the tokens *er* and *est* are meaningful suffixes. *new* is not merged into one token and *id* in *wider* and *widest* has no linguisitic meaning.
- Meaningful morphemes can be useful for downstream NLP tasks because they can help understand word meanings and grammatical forms.
- However BEP does not always recognize true morphemes since it merges tokens based on frequency rather than meaning. This can result in negativ/false performance in downstream NPL tasks.
- For example with the word [w id er] the model may need more training examples to learn the meaningful connection between the tokens or it might also have difficulty recognizing the word's meaning. In a text classification task this could lead to lower accuracy.

```
# Task 2a - Rule-based stemming

# define rules for English:
# remove suffixes:
# ing    playing -> play
# ed     played -> play
# ly     quickly -> quick
# es     moxes -> box
# s      cats -> cat
# remove prefixes:
# un    unhappy -> happy
# dis   disagree -> agree
# mis   misunderstand -> understand (stemmimng only applied once for
prefix)

#define rules for German:
# remove suffixes:
# en    zahlen -> zahl
# keit  Heiterkeit -> Heiter
# e     Hunde -> e
# st    kannst -> kann
# s     Autos -> Auto
#remove prefixes:
# un    unglücklich -> glücklich (lich could also be removed with a rule)
# ver   verkaufen -> kaufen
# ge    gesagt -> sagt

# test words English:
# playing (true), played (true), cats (true), boxes (true), byes (false),
dogs(true),
# his (false), walking(true), studies (true), sing(false), news(false),
# misunderstand (true), disagree (true), disturb (false), mister (false)
# unhappy (true), until (false)

# test words German:
# unglücklich (true), verkaufen (true), gesagt (true), zahlen (true), Hunde
(true),
# Heiterkeit (true), Autos (true), kannst (true), unklar (true), Gesicht
(false),
# gestern (false), Ungeheuer (false), Universität (false), Keks (false),
# Verb (false), verbinden(true), Ast (false), Keule (false), verkaufst
(true)


# Define stemming rules
# from long to short
prefixes_english = ["dis", "mis", "un"]
suffixes_english = ["ing", "ed", "es", "ly", "s"]

prefixes_german = ["ver", "ge", "un", ]
suffixes_german = ["keit", "en", "st", "e", "s"]
```

```
def stem(word, language):
    """cuts prefixes ond suffixes depending on language"""
    if language == "english":
        prefixes = prefixes_english
        suffixes = suffixes_english
    elif language == "german":
        prefixes = prefixes_german
        suffixes = suffixes_german
    else:          # other languages could be added later
        return word

    # remove prefix
    for prefix in prefixes:
        if word.lower().startswith(prefix): # lower for German words
            word = word[len(prefix):]
            break # only one rule applyed per word

    # Remove suffixes
    for suffix in suffixes:
        if word.lower().endswith(suffix):
            word = word[:-len(suffix)]
            break

    return word

#test English
words_english = ["playing", "played", "walking", "sing", "quickly", "cats",
"dogs", "his",
         "news", "boxes", "byes", "studies", "misunderstand", "disagree",
"disturb",
         "mister", "unhappy", "until"]

words_german = [
    "unglücklich", "verkaufen", "gesagt", "zahlen", "Hunde", "Heiterkeit",
"Autos",
    "kannst", "unklar",  "Gesicht", "gestern", "Ungeheuer", "Universität",
"Keks",
    "Verb", "verbinden", "Ast", "Keule", "verkaufst"
]

# Apply English rules
print("\nApply on test words")
print("English stemming:")
for word in words_english:
    print(word, "->", stem(word, "english"))

# Apply German rules
print("\nGerman stemming:")
for word in words_german:
    print(word, "->", stem(word, "german"))


# Second part
print("\nApply on sentences")
```

```
# test sentences
sentences_english = ["The children are playing with their dogs.",
                     "She quickly opened the boxes.",
                     "The unhappy student disagreed with the teacher.",
                     "They are walking and singing in the garden.",
                     "He misunderstood the question and replayed the
video."]

sentences_german = ["Die Kinder spielen mit ihren Hunden.",
                    "Der unglückliche Schüler verkauft seine Bücher.",
                    "Sie bezahlten gestern die Einkäufe.",
                    "Die Verkäufer arbeiten meistens sehr schnell.",
                    "Das Geschenk wurde gestern ausgepackt und geöffnet."]

# Apply English stemming
print("English sentences:")
for sentence in sentences_english:
    words = sentence.split()
    stemmed_words = [stem(word.lower().strip(".,!?"), "english")
                     for word in words]
    print(sentence)
    print(stemmed_words)


# Apply German stemming
print("\nGerman sentences:")
for sentence in sentences_german:
    words = sentence.split()
    stemmed_words = [stem(word.lower().strip(".,!?"), "german")
                     for word in words]
    print(sentence)
    print(stemmed_words)```
```

assignment-02\task-2b-bpe.py

```python
# Task 2 b) Byte Pair Encoding (BPE)

'''
Genutzte Ressourcen:
Umsetzung Code und Programmstruktuer:
https://towardsdatascience.com/byte-pair-encoding-for-beginners-708d4472c0c7/

Fachliche und wissenschaftliche Quelle
Rico Sennrich, Barry Haddow, and Alexandra Birch.
2016. Neural Machine Translation of Rare Words with Subword Units.
arXiv preprint arXiv:1508.07909v5. https://arxiv.org/abs/1508.07909

'''

import collections
import re
import json

# Anzahl Iterationen
num_merges = 15
corpus_file = "assignment-02\\corpora-de.json"
unknown_file = "assignment-02\\unknown-de.txt"

def read_corpora(input_file):
    vocab = {}
    with open(input_file, "r", encoding="utf-8") as file:
        vocab = json.load(file)
        return vocab

def read_unknown(input_file):
    with open(input_file, "r", encoding="utf-8") as file:
        unknown_text = file.read()
        return unknown_text


# Häufigkeit benachbarter (Trenner=" ") Elemente zählen
def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        elements = word.split()
        for i in range(len(elements)-1): # -1, weil i noch einen Nachfolger haben muss.
            pairs[elements[i], elements[i+1]] += freq
    return pairs

# Benachbarte Elemente verschmelzen, basierend auf den Elementen im Korpus
def merge_vocab(pair, vocab_in):
    vocab_out = {}
    # Suchtext
    search_text = ' '.join(pair)
    # Ersatztext
    replace_text = ''.join(pair)
```

```
52      # Sonderzeichen maskieren
53      escaped_search_text = re.escape(search_text)
54      # Suchmuster
55      pair_pattern = re.compile(r'(?<!\S)' + escaped_search_text + r'(?!\S)')
56
57      for word, frequency in vocab_in.items():
58          new_word = pair_pattern.sub(replace_text, word)
59          vocab_out[new_word] = frequency
60
61      print(f"vocab_out {vocab_out}")     # debug print
62      return vocab_out
63
64  # Beste Paarung, zuerst im Alphabet
65  def best_alphabetical(pairs):
66      # Paar mit höchtem Wert = Anzahl
67      max_count = max(pairs.values())
68      if max_count == 1:
69          print("ACHTUNG! Nur noch je 1 Paar vorhanden!")
70      # Alle Paare mit höchstem Wert finden
71      all_bests = []
72      all_bests_concat = []
73      for pair in pairs:
74          if pairs[pair] == max_count:
75              all_bests.append(pair)
76              all_bests_concat.append("".join(pair))
77      # Erstes Paar im Alphabet finden
78      best_index = all_bests_concat.index(min(all_bests_concat))
79      best = all_bests[best_index]
80      print(f"max_count: {max_count} all_bests: {all_bests} best: {best}") # debug print
81      return best, all_bests
82
83  # Token-Liste
84  def current_token_list(vocab):
85      vocab_list = list(vocab)
86      current_list = []
87      for word in vocab_list:
88          tokens = word.split()
89          for token in tokens:
90              if token not in current_list:
91                  current_list.append(token)
92      return current_list
93
94  def learn_bpe(vocab, num_merges):
95      for i in range(num_merges):
96          print()         # debug print
97          print("i:", i)  # debug print
98          # Häufigkeit benachbarter, durch " " getrennte Elemente, zählen
99          pairs = get_stats(vocab)
100         # Häufigste Paarung (alphabetisch) in best ablegen
101         best, all_bests = best_alphabetical(pairs)
102         # Häufigsten Elemente verschmelzen, gibt das Ergebnis als neuen Korpus zurück
103         vocab = merge_vocab(best, vocab)
104         token_list = current_token_list(vocab)
105         print(f"token_list {token_list}")
```

```
    return vocab, token_list



# Mit KI-Unterstützung generierter Code -- Anfang

# Ein Wort mit Hilfe der Token-Liste in Tokens zerlegen
def segment_word(word, token_list):
    tokens = []      # hier sammeln wir die gefundenen Tokens
    position = 0     # aktuelle Stelle im Wort

    # So lange wiederholen, bis das ganze Wort zerlegt ist
    while position < len(word):
        found_token = None

        # Vom längstmöglichen Stück zum kürzesten probieren
        for end in range(len(word), position, -1):
            piece = word[position:end]
            if piece in token_list:
                found_token = piece
                break    # längstes passendes Token gefunden, Suche beenden

        # Kein Token passt: einzelnes Zeichen als Token nehmen
        if found_token is None:
            found_token = word[position]

        tokens.append(found_token)
        position = position + len(found_token)   # hinter dem Token weitermachen

    return tokens


# Einen ganzen Korpus (Text) segmentieren
def segment_corpus(text, token_list):
    result = []
    words = text.split()            # Text an Leerzeichen in Wörter teilen
    for word in words:
        word_tokens = segment_word(word, token_list)
        result.append(" ".join(word_tokens))   # Tokens mit Leerzeichen verbinden
    return result
# Mit KI-Unterstützung generierter Code -- Ende



# Hauptroutine
# Token-Liste, basierend auf Korpus erstellen
print(f"num_merges: {num_merges}")            # debug print
vocab = read_corpora(corpus_file)
print(f"corpus: {vocab}")
vocab, final_token_list = learn_bpe(vocab, num_merges)
print()                                       # debug print
print(f"final_token_list {final_token_list}") # debug print

# BPE Encoding auf unbekannten Korpus anwenden
```

```
160 # Beispielaufruf
161 unknown_text = read_unknown(unknown_file)
162 segmented = segment_corpus(unknown_text, final_token_list)
163 print(f"segmented: {segmented}")
164
165
```

Task 2 output and reflection

2a Results for rule-based stemming

Apply on test words

English stemming:

playing -> play

played -> play

walking -> walk

sing -> s

quickly -> quick

cats -> cat

dogs -> dog

his -> hi

news -> new

boxes -> box

byes -> by

studies -> studi

misunderstand -> understand

disagree -> agree

disturb -> turb

mister -> ter

unhappy -> happy

until -> til

German stemming:

unglücklich -> glücklich

verkaufen -> kauf

gesagt -> sagt

zahlen -> zahl

Hunde -> Hund

Heiterkeit -> Heiter

Autos -> Auto

kannst -> kann

unklar -> klar

Gesicht -> sicht

gestern -> stern

Ungeheuer -> geheuer

Universität -> iversität

Keks -> Kek

Fenster -> Fenster

Verb -> b

verbinden -> bind

Ast -> A

Keule -> Keul

verkaufst -> kauf

Apply on sentences

English sentences:

The children are playing with their dogs.

['the', 'children', 'are', 'play', 'with', 'their', 'dog']

She quickly opened the boxes.

['she', 'quick', 'open', 'the', 'box']

The unhappy student disagreed with the teacher.

['the', 'happy', 'student', 'agre', 'with', 'the', 'teacher'] -> opposite meaning now

They are walking and singing in the garden.

['they', 'are', 'walk', 'and', 'sing', 'in', 'the', 'garden']

He misunderstood the question and replayed the video.

['he', 'understood', 'the', 'question', 'and', 'replay', 'the', 'video']

German sentences:

Die Kinder spielen mit ihren Hunden.

['di', 'kinder', 'spiel', 'mit', 'ihr', 'hund']

Der unglückliche Schüler verkauft seine Bücher.

['der', 'glücklich', 'schüler', 'kauft', 'sein', 'bücher'] -> now opposite Meaning

Sie bezahlten gestern die Einkäufe.

['si', 'bezahlt', 'stern', 'di', 'einkäuf']

Die Verkäufer arbeiten meistens sehr schnell.

['di', 'käufer', 'arbeit', 'meisten', 'sehr', 'schnell']

Das Geschenk wurde gestern ausgepackt und geöffnet.

['da', 'schenk', 'wurd', 'stern', 'ausgepackt', 'd', 'öffnet']

Task 2 b) Byte Pair Encoding (BPE)
Task 2.2 Select two different corpora

English Corpus

{"test": 1, "words": 1, "English": 1, "playing": 1, "played": 1, "cats": 1, "boxes": 1, "byes": 1, "dogs": 1, "his": 1, "walking": 1, "studies": 1, "sing": 1, "news": 1, "misunderstand": 1, "disagree": 1, "disturb": 1, "mister": 1, "unhappy": 1, "until": 1}

Merged corpora after 16 Turns

{'test': 1, 'words': 1, 'English': 1, 'playing': 1, 'played': 1, 'cats': 1, 'boxes': 1, 'byes': 1, 'dogs': 1, 'his': 1, 'walking': 1, 'studies': 1, 'sing': 1, 'news': 1, 'misunderstand': 1, 'disagree': 1, 'disturb': 1, 'mister': 1, 'unhappy': 1, 'until': 1}

After 16 iterations, only pairs of tokens, each occurring once in the training vocabulary, remained. Further iterations generated individual words again, until the entire corpus was once more represented as individual words in the token list. With this method, it appears to be important to define a suitable termination condition for token generation.

Final token list after 16 iterations

['t', 'es', 'w', 'o', 'r', 'd', 's', 'Engl', 'is', 'h', 'play', 'ing', 'e', 'c', 'a', 'b', 'x', 'y', 'g', 'l', 'k', 'st', 'u', 'i', 'n', 'mis', 'un', 'er', 'dis', 'p']

Application of the BPE model to an unknown text with language match (English – English)

The children are playing with their dogs.
['T h e', 'c h i l d r e n', 'a r e', 'play ing', 'w i t h', 't h e i r', 'd o g s '.]

She quickly opened the boxes.
['S h e', 'q u i c k l y', 'o p e n e d', 't h e', 'b o x es .']

The unhappy student disagreed with the teacher.
['T h e', 'un h a p p y', 'st u d e n t', 'dis a g r e e d', 'w i t h', 't h e', 't e a c h er .']

They are walking and singing in the garden.
['T h e y', 'a r e', 'w a l k ing', 'a n d', 's ing ing', 'i n', 't h e', 'g a r d e n .']

He misunderstood the question and replayed the video.
, ['H e', 'mis un d er st o o d', 't h e', 'q u es t i o n', 'a nd', 'r e play e d', 't h e', 'v i d e o .']

Language Mitsmatch English training – German unknown text:

segmented: ['D i e', 'K i n d er', 's p i e l e n', 'm i t', 'i h r e n', 'H un d e n .', 'D er', 'un g l ü c k l i c h e', 'S c h ü l er', 'v er k a u f t', 's e i n e', 'B ü c h er .', 'S i e', 'b e z a h l t e n', 'g es t er n', 'd i e', 'E i n k ä u f e .', 'D i e', 'V er k ä u f er', 'a r b e i t e n', 'm e is t e n s', 's e h r', 's c h n e l l .', 'D a s', 'G es c h e n k', 'w u r d e', 'g es t er n', 'a u s g e p a c k t', 'un d', 'g e ö f f n e t .']

German Corpus

{ "u n g l ü c k l i c h": 1, "v e r k a u f e n": 1, "g e s a g t": 1, "z a h l e n": 1, "H u n d e": 1, "H e i t e r k e i t": 1, "A u t o s": 1, "k a n n s t": 1, "u n k l a r": 1, "G e s i c h t": 1, "g e s t e r n": 1, "U n g e h e u e r": 1, "U n i v e r s i t ä t": 1, "K e k s": 1, "F e n s t e r": 1, "V e r b": 1, "v e r b i n d e n": 1, "A s t": 1, "K e u l e": 1, "v e r k a u f s t": 1}

Merged corpus after 18 Turns

{'un g l ü c kl ich': 1, 'ver kauf en': 1, 'ge s a g t': 1, 'z a h l en': 1, 'H un d e': 1, 'H eit er k eit': 1, 'A u t o s': 1, 'ka n n st': 1, 'un kl a r': 1, 'G e s ich t': 1, 'ge ster n': 1, 'Un ge h e u er': 1, 'Un i ver s it ä t': 1, 'Ke k s': 1, 'F en ster': 1, 'V er b': 1, 'ver b i n d en': 1, 'A st': 1, 'Ke u l e': 1, 'ver kauf st': 1}

In the German corpus, after 18 passes, only token pairs each occurring once remained.

Final token list after 18 turns

['un', 'g', 'l', 'ü', 'c', 'kl', 'ich', 'ver', 'kauf', 'en', 'ge', 's', 'a', 't', 'z', 'h', 'H', 'd', 'e', 'eit', 'er', 'k', 'A', 'u', 'o', 'ka', 'n', 'st', 'r', 'G', 'ster', 'Un', 'i', 'it', 'ä', 'Ke', 'F', 'V', 'b']

Application of the BPE model to an unknown text with language match (German – German)

Die Kinder spielen mit ihren Hunden.
['D i e', 'K i n d er', 's p i e l en', 'm it', 'i h r en', 'H un d en .']
Der unglückliche Schüler verkauft seine Bücher.
['D er', 'un g l ü c kl ich e', 'S c h ü l er', 'ver kauf t', 's e i n e', 'B ü c h er .']
Sie bezahlten gestern die Einkäufe.
['S i e', 'b e z a h l t en', 'ge ster n', 'd i e', 'E i n k ä u f e .']
Die Verkäufer arbeiten meistens sehr schnell.
['D i e', 'V er k ä u f er', 'a r b eit en', 'm e i st en s','s e h r', 's c h n e l l .']
Das Geschenk wurde gestern ausgepackt und geöffnet.
['D a s', 'G e s c h en k', 'w u r d e', 'ge ster n', 'a u s ge p a c k t', 'un d', 'ge ö f f n e t .']

Language Mitsmatch German training – English unknown text:

segmented: ['T h e', 'c h i l d r en', 'a r e', 'p l a y i n g', 'w it h', 't h e i r', 'd o g s .', 'S h e', 'q u i c kl y', 'o p en e d', 't h e', 'b o x e s .', 'T h e', 'un h a p p y', 'st u d en t', 'd i s a g r e e d', 'w it h', 't h e', 't e a c h er .', 'T h e y', 'a r e', 'w a l k i n g', 'a n d', 's i n g i n g', 'i n', 't h e', 'g a r d en .', 'H e', 'm i s un d er st o o d', 't h e', 'q u e st i o n', 'a n d', 'r e p l a y e d', 't h e', 'v i d e o .']

<u>Overall interpretation Task 2:</u>

1. <u>Cross linguistic mismatch:</u>

- If you apply a German BPE model to English sentences (or vice versa), words are split into unnaturally tiny fragments or individual characters.
- The model cannot recognize typical prefixes, suffixes, or root words of the other language because its learned rules (merges) are based on the wrong statistical patterns.
- A sentence requires significantly more tokens in a mismatch scenario, which reduces processing efficiency.

2. <u>Rule-Based Stemming</u>

Strengths:

- It seems easy to emplement and relativly quick to execute. The computational costs seem relativly low.
- Cuts words down to stems. Good for Information Retrieval

Weaknesses

- Creates unnatural word forms and stems without linguistic meaning
- Cannot handle foreign words or grammar exceptions outside its predefined rules

3. Byte Pair Encoding (BPE):
   Strengths:

- can technically process any unknown word by falling back to smaller subwords or characters – (not considering the quality)
- adapts to frequency patterns without needing manual grammar rules

Weaknesses:

- Quality relies on the size and language of the training data
- performs poorly when applied to unseen languages